

chart.js:6

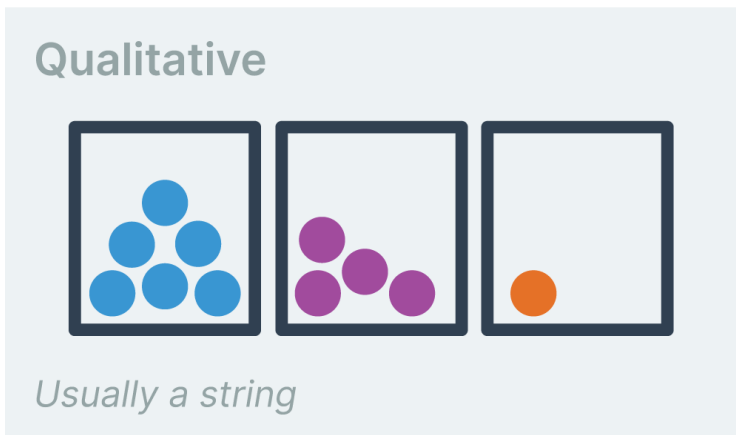
(index)	Value
time	1514782800
summary	"Clear throughout the day."
icon	"clear-day"
sunriseTime	1514809280
sunsetTime	1514842810
moonPhase	0.48
precipIntensity	0
precipIntensityMax	0
precipProbability	0
temperatureHigh	18.39
temperatureHighTime	1514836800
temperatureLow	12.23
temperatureLowTime	1514894400
apparentTemperatureHigh	17.29
apparentTemperatureHighTime	1514844000
apparentTemperatureLow	4.51
apparentTemperatureLowTime	1514887200
dewPoint	-1.67
humidity	0.54
pressure	1028.26
windSpeed	4.16
windGust	13.98
windGustTime	1514829600
windBearing	309
cloudCover	0.02
uvIndex	2
uvIndexTime	1514822400
visibility	10
temperatureMin	6.17
temperatureMinTime	1514808000
temperatureMax	18.39
temperatureMaxTime	1514836800
apparentTemperatureMin	-2.19
apparentTemperatureMinTime	1514808000
apparentTemperatureMax	17.29
apparentTemperatureMaxTime	1514844000
date	"2018-01-01"

► Object

Our dataset

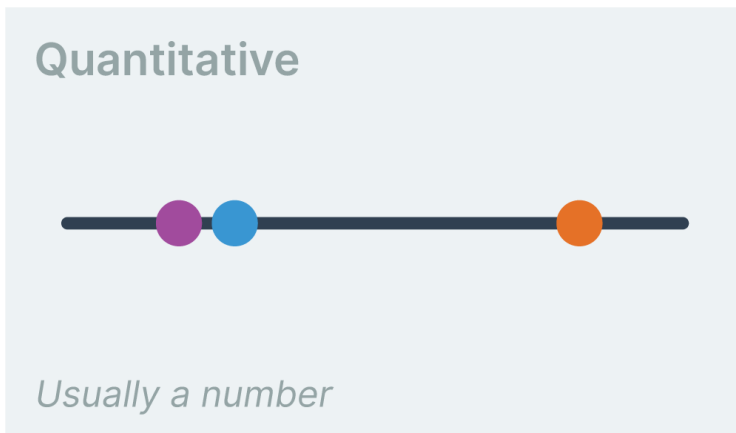
There are many different values here, but two basic types: **strings** and **numbers**. These two types can roughly be split (respectively) into two basic types of data: **qualitative** and **quantitative**.

Qualitative data (our strings) does not have a numerical value, but it can be put into categories. For example, `precipType` can either have a value of “rain” or “snow”.



data type - qualitative

Quantitative data (our numbers) is numerical and can be measured objectively. For example, `temperatureMax` has values ranging from 10°F to 100°F.

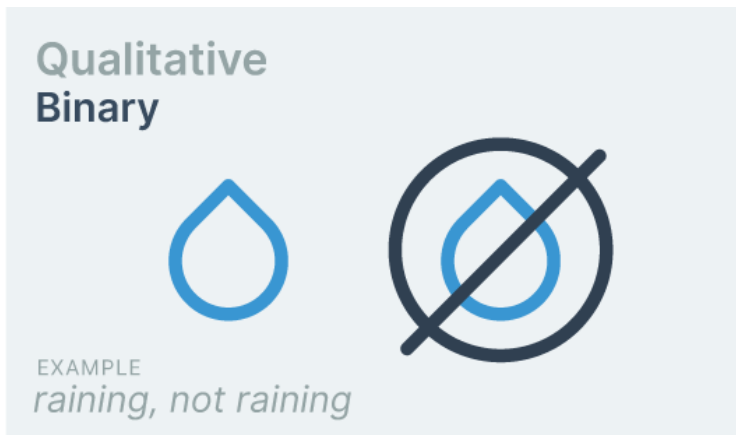


data type - quantitative

Both of these types of data can be broken down even further.

Qualitative Data

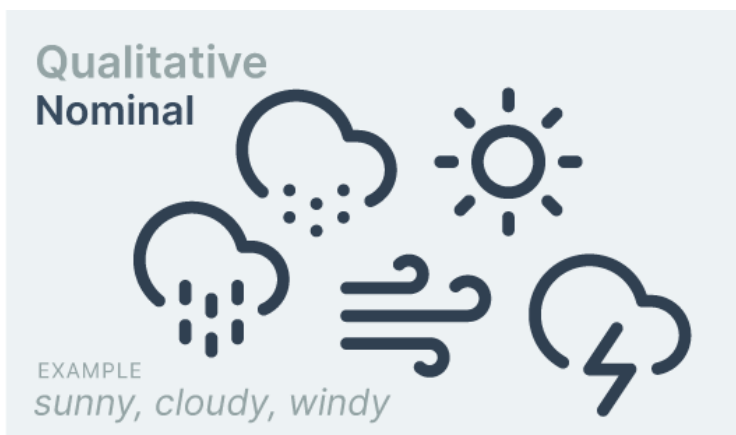
Binary data can be placed into only two categories.



data type - binary

For example, if our weather data had an **did rain** metric that was either **true** or **false**, that metric would be binary.

Nominal data can be placed multiple categories that don't have a natural order.



data type - nominal

For example, our weather data has the metric **icon** with values such as **clear-day** and **wind** — these values can't be ordered.

Ordinal data can be placed in multiple categories with a natural order.



data type - ordinal

For example, if our weather data instead represented **wind speed** values with not windy, somewhat windy, and very windy, that metric would be ordinal.

Quantitative Data

Discrete data has numerical values that can't be interpolated between, such as a metric that can only be represented by an integer (whole number).



data type - discrete

A classic example is **number of kids** — a family can have 1 or 2 kids, but not 1.5 kids. With weather data, a good example would be number of tornados that happened.

Continuous data has numerical values that can be interpolated between. Usually a metric will fall under this category — for example, **max temperature** can be 50°F or 51°F or 50.5°F.



data type - continuous

This categorization is just one way to group types of data. There are other ways, but this categorization is common and a handy place to start. When you are looking at a new dataset, familiarize yourself by categorizing each metric.

Ways to visualize a metric

Now that we understand the metrics that we're working with, we need to decide how to represent them visually. Keep in mind that we, as humans, are able to judge some dimensions more quickly or precisely than others.

Let's go through some possible ways we could represent **temperature**, as an example.

Size

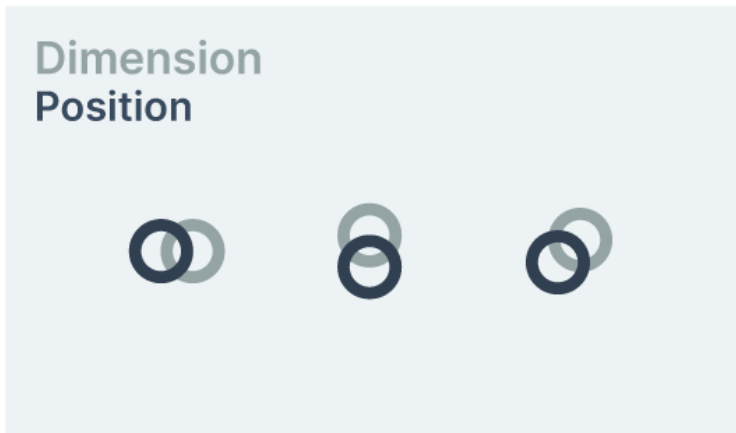


dimension - size

We could represent **temperature** by sizing an element: larger for higher temperatures and smaller for lower temperatures. We could also scale just the height or the width — when making our histograms, we created taller bars for higher numbers.

Humans are very fast and precise at perceiving size differences (especially height or width) between objects, making it a good choice when the exact differences matter.

Position

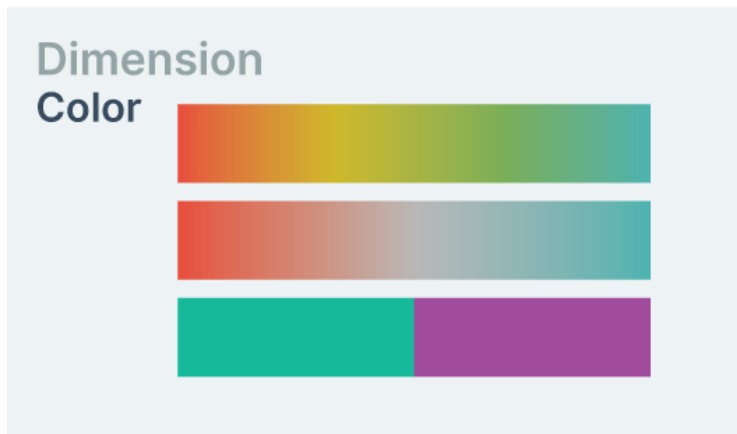


dimension - position

We could represent **temperature** by moving an element: horizontally, vertically, even diagonally. For example, we could make a scatter plot with lower temperatures to the left and higher temperatures to the right.

Like size, humans are very fast and precise at comparing horizontal and vertical positions, making it a great choice when precision matters.

Color



dimension - color

We could represent **temperature** by giving our elements a different color. For example, we could make a scatter plot with dots colored from blue (lower temperatures) to red (higher temperatures). Choosing colors can be overwhelming — don't worry, we have a section to help with just that coming up in this Chapter.

Humans are less adept at distinguishing two colors than distinguishing two sizes or two positions. However, we can more easily average colors together, making color a good choice for visualizations where the overall picture matters. If both the specific values and the overall picture are important, try showing the overall picture and progressively surfacing exact values with tooltips or a companion table.

There are other dimensions (for example, **orientation** or **pattern**), but these basic ones will get you thinking on the right path. These dimensions might remind you of a concept we've been using: **scales**. Scales have been helping us translate data values into a physical dimension: for example, **temperature** into **height of a bar in pixels**.

When you have a dataset that is chock full of metrics like our weather data, it often isn't ideal to visualize all of it in one chart. Focused charts are the most effective — sticking with 2-4 dimensions will help the user focus on what's important without being overwhelmed.

Putting it together

It can be tempting to jump right in and say “Oh I know, I’m going to make a bar chart!” But let’s investigate further: what is a bar chart made out of? A bar chart is made up of **bars** that vary in size (**height**) and are spread out horizontally (**position**).

In the next chapter, we’ll talk about common types of charts and how the **type of metrics** and **ways to visualize them** apply to each. But there is a whole world of chart types that haven’t yet been created! Now that you have this framework, you’ll be able to look at a dataset and brainstorm unique ways to visualize it.

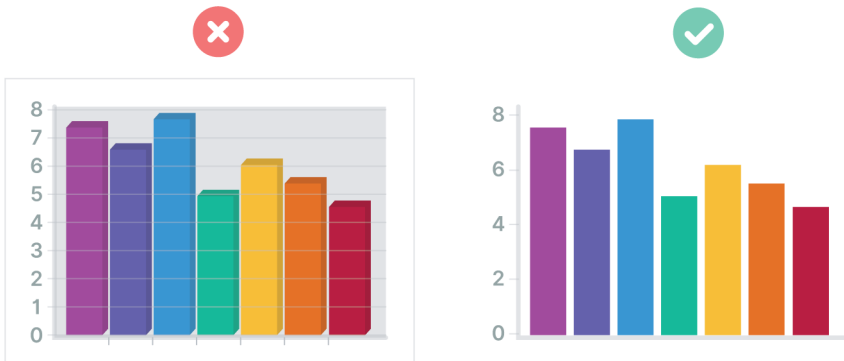
Chart design

Now that you understand these basics, you have the tools you need to make effective and intuitive charts. Once you’ve decided on the what and the how, here are some tips for keeping those charts easy to read.

Simplify, simplify, simplify

Many chart libraries will include default styles in order to satisfy as many use cases as possible. This can create charts with more elements than are necessary. Fortunately, we don’t need to rely on a library and can strip away as much as possible.

After you finish creating a chart, give it a critical look and ask yourself, “Is everything on here necessary?” If the answer is no, remove the cruft! No need to overwhelm your reader.



Simplified chart

Annotate in-place

It can be tempting to throw a legend next to a chart to clarify a color or size scale. While we should make sure to explain every part of our chart, ask yourself if you can label these elements directly. In-place annotations put the description as close as possible, which prevents forcing the reader to look back and forth between elements.

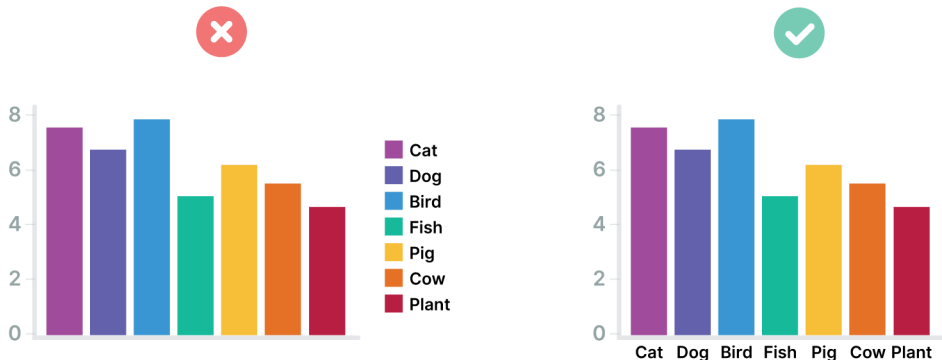


Chart with labels

Add enhancements, but not too many

As a corollary to the last two tips, check to see if you can enhance your chart to help create insights. For example, if the goal of your chart is to show trends, consider adding a trend or reference line to make any patterns more apparent.

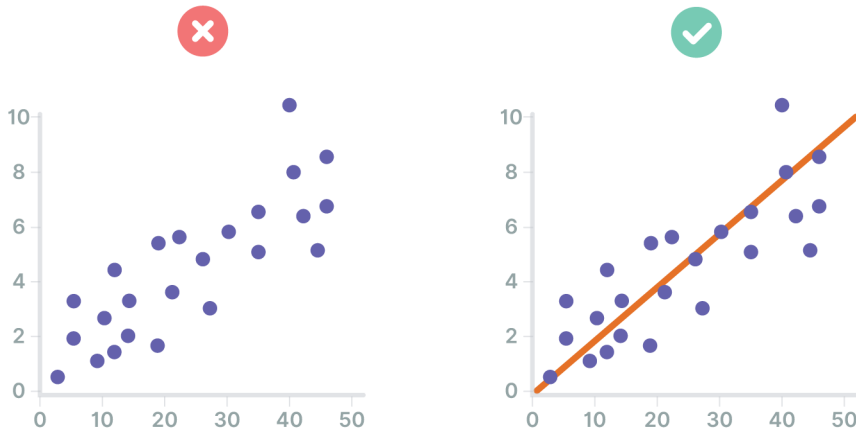


Chart with trend line

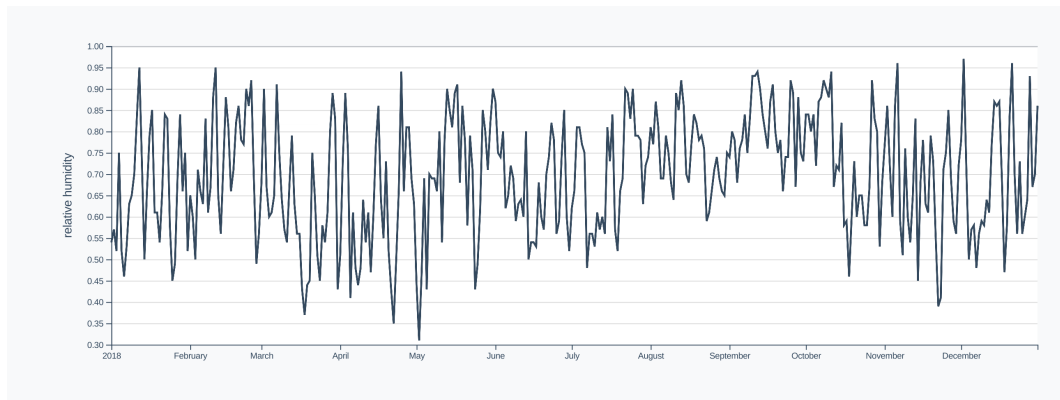
Example redesign

Let's redesign a simple chart to see these tips in practice. The goal of this chart is to **examine how humidity changes, depending on the time of year.**

We've decided to visualize this data using a timeline — open it up at <http://localhost:8080/07-data-visualization-basics/humidity-timeline/draft/>⁵⁴. We'll slowly work towards the finished version at <http://localhost:8080/07-data-visualization-basics/humidity-timeline/completed/>⁵⁵. Don't peek yet!

⁵⁴<http://localhost:8080/07-data-visualization-basics/humidity-timeline/draft/>

⁵⁵<http://localhost:8080/07-data-visualization-basics/humidity-timeline/completed/>



Humidity timeline

For practice, take a minute and write down the ways you would improve on this chart, with our goal in mind.

Goal: examine how humidity changes, depending on the time of year

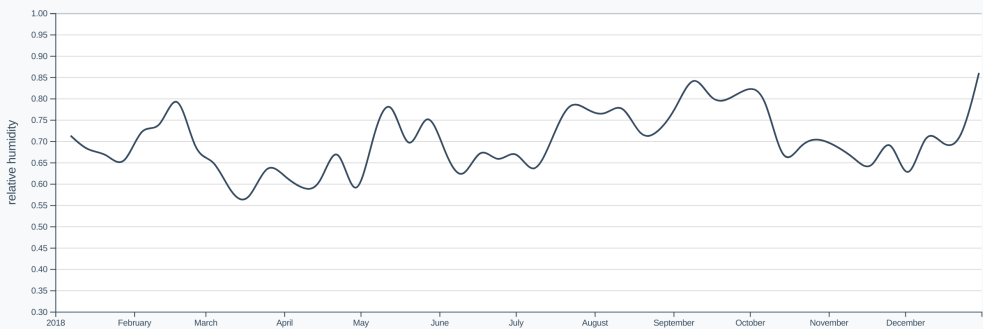
Ready to dive in? There is no “correct” way to improve on this chart, but we’ll do our best to make it easier to digest quickly.

The first problem we’ll tackle is the noisiness of the data. While the daily dips and spikes may help answer other questions, they only distract from our goal of investigating seasonal trends. Let’s downsample the data to one point per week and smooth our line using `d3.curveBasis()`.

If you’re following along, we have a `downsampleData()` function at the bottom of the file that we can pass our `dataset`, `xAccessor`, and `yAccessor` and receive a downsampled dataset with weekly values.

`d3.line()` has a method `.curve()` that can be passed a interpolation function. The default interpolation is linear: the line connects points directly. For a smoother line, check out the different interpolation options built in to d3 in the [d3-shape documentation](https://github.com/d3/d3-shape#curves)^a. Here, we're adding the code `.curve(d3.curveBasis)` to diminish the importance of specific data points and give the reader an idea of the overall line shape.

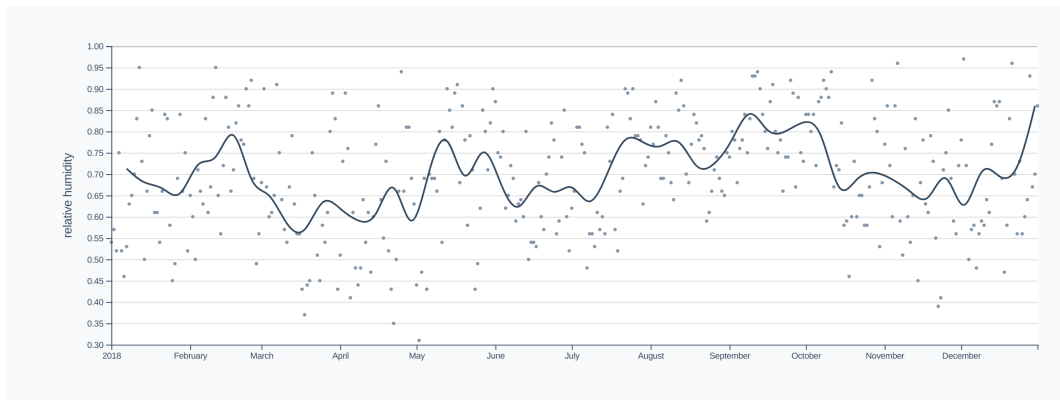
^a<https://github.com/d3/d3-shape#curves>



Humidity timeline, smoothed

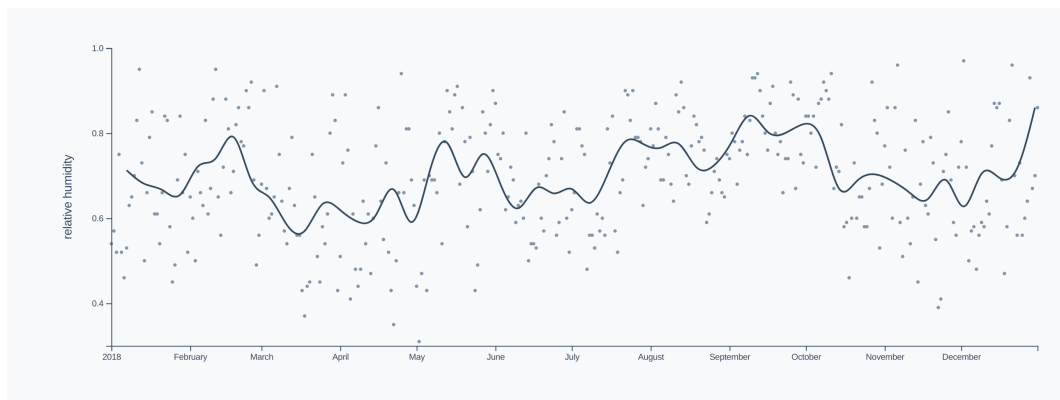
Let's add the original points back in, in the form of small circles. We don't want to lose the granularity of the original data, even when we're getting the basic *trend* with the downsampled line.

To prevent from drawing attention away from our trend line, we'll make these dots a lighter grey.



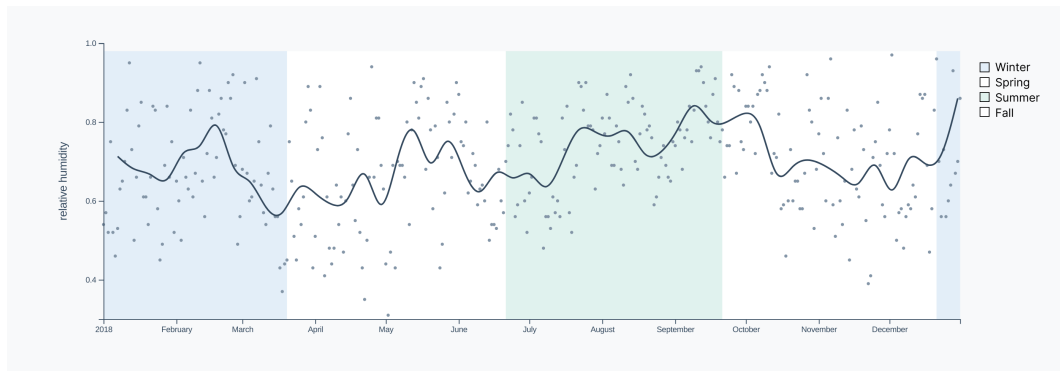
Humidity timeline, smoothed with dots

There's a lot going on right now! Let's simplify a bit and take out the grid marks and chart background. We'll also specify a smaller number of ticks (`.ticks(3)`) for our y axis — the extra ticks aren't giving any useful information since readers can easily extrapolate between values.



Humidity timeline, simpler

Revisiting our goal, we realize that we want to focus on trends based on the time of year. We're showing the months on our x axis, but we can do some work for the reader and highlight the different seasons. Let's block out each season with a `<rect>` underneath our main data elements.

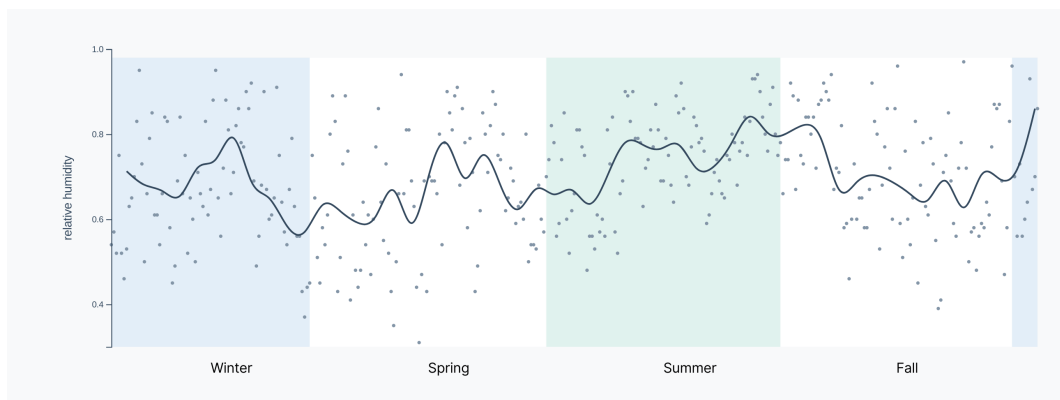


Humidity timeline, with seasons

While the legend is helpful, there are a few issues with it.

1. Both Spring and Fall are white because they are transitional seasons. This makes it unclear which is which on the chart.
2. As we discussed earlier in the chapter, legends make the reader look back and forth between the chart and the legend.

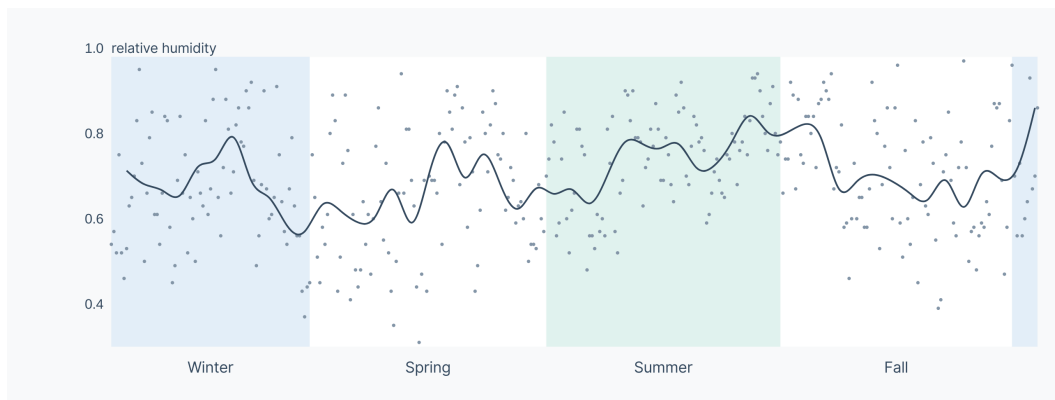
To fix these issues, let's label the seasons directly on the chart instead of having an x axis.



Humidity timeline, with season labels

Let's focus on the y axis. Rotated y axis labels are great — they're often the only way to fit the label to the side of our chart. However, the label is divorced from the values, and rotating text makes it harder to read.

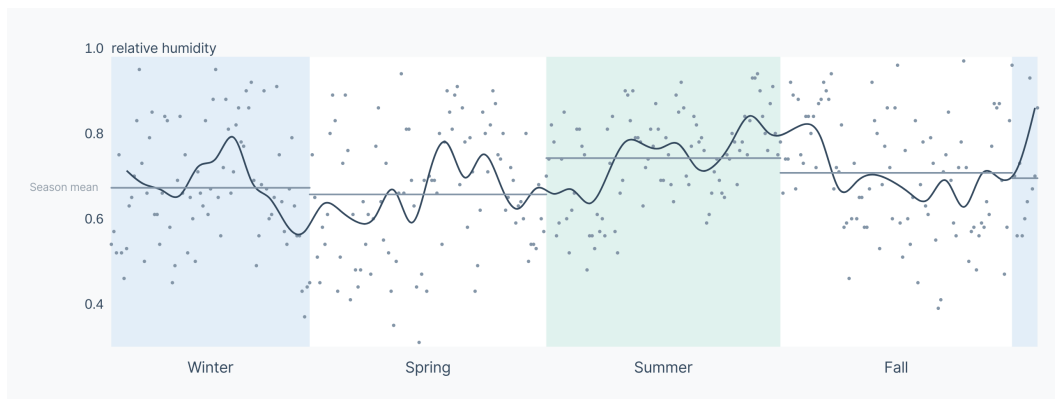
Instead, let's signify the units of our y axis by incorporating it into a phrase with our top y tick value. Human-readable labels can do some of the digesting work a reader has to do, hinting at how to interpret a number.



Humidity timeline, with inline y label

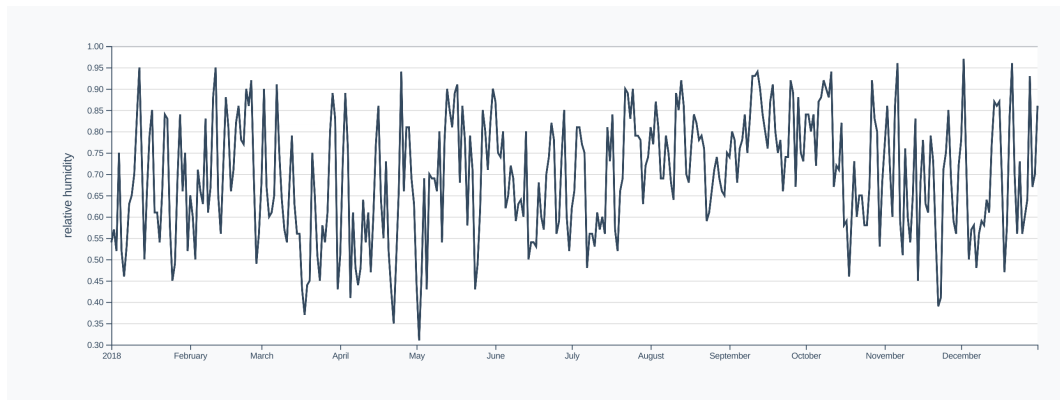
While we've made it easier to compare trends across seasons, it's not easy to conclude how the seasons compare in general. We *could* downsample our data and only display one point per season, but then we'd lose any insights from seasonal trends.

Instead, let's add seasonal means as lines, which should enhance the chart but not take away from the main picture.

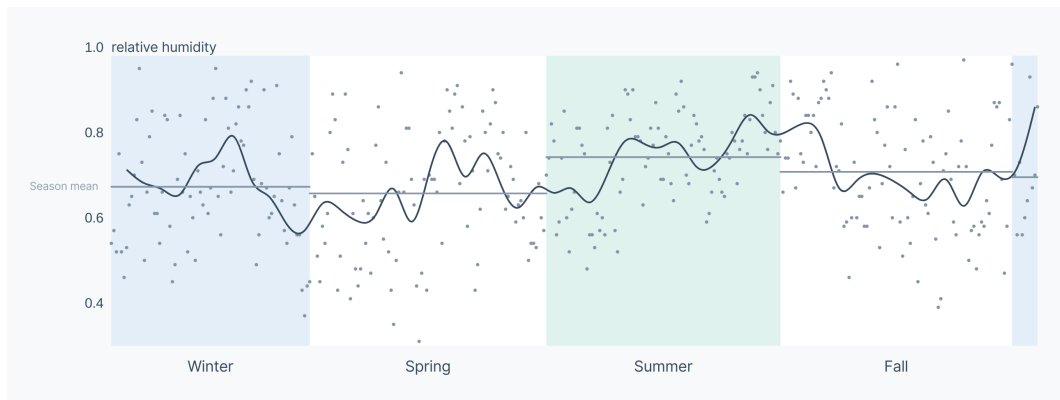


Humidity timeline, with means

Nice! Now we can easily see that the humidity is low in the Winter, but not as low as in the Spring. Let's look at two charts side-by-side.



Humidity timeline, start



Humidity timeline, finish

While both charts display the same data and can both be helpful, the final version is much more effective for our goal: to **examine how humidity changes, depending on the time of year**.

Colors

One of the hardest parts of creating charts is choosing colors. The wrong colors can make a chart unappealing, ineffective, and, worst of all, impossible to read. This section will give you a good framework for choosing colors, important facts about human perception, and simple tips.

Color scales

When choosing a color scale, you first want to identify its purpose. Going back to what we know about data types, there are three basic use cases:

1. Representing a category
2. Representing a continuous metric
3. Representing a diverging metric

Let's dive deeper into each of these.

1. Representing a category

The first two data types we talked about (**binary** and **nominal**) will be best represented with a categorical color scheme.

Since our metric values don't have a natural order, we don't want to use a color scheme that has a natural order (like white to black).

d3 has built-in color schemes in its **d3-scale-chromatic**⁵⁶ library. We can see them listed under **Categorical** if we start our server (**live-server**) and navigate to <http://localhost:8080/07-data-visualization-basics/scales/>⁵⁷ in our browser.



color scales - categorical

These **categorical** color schemes have been carefully designed to have enough contrast between colors. Each of these schemes is an array of colors — for example, to use the first scale in this list, we would access each color at its index in `d3.schemeCategory10`.

⁵⁶<https://github.com/d3/d3-scale-chromatic>

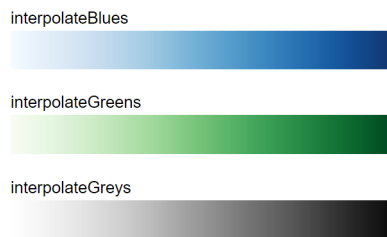
⁵⁷<http://localhost:8080/07-data-visualization-basics/scales/>

For **categorical** color schemes, it's helpful for each color to have a different descriptive name. For example, there might be two colors that could be described as “blue”, which can be confusing to talk about.

2. Representing a continuous metric

For metrics that are **continuous**, we'll want a way to interpolate in between color values. For example, we could represent **humidity** values with a color scale ranging from white to dark blue.

d3-scale-chromatic⁵⁸'s built-in continuous scales are visible under the **Sequential** section.

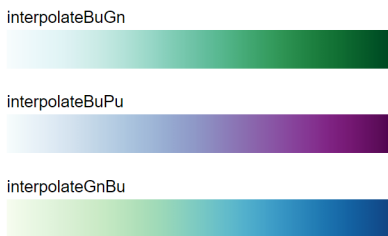


color scales - sequential

These are color scales instead of color schemes — `d3.interpolateBlues()` is a function instead of an array of colors. To get a color, we can give `d3.interpolateBlues()` a decimal in between 0 and 1 — 0 would return the leftmost color (a light gray) and 1 would give us the dark blue on the right. To put it in familiar terms, `d3.interpolateBlues()` is a scale with a domain of `[0,1]` and a range of `[light gray, dark blue]`.

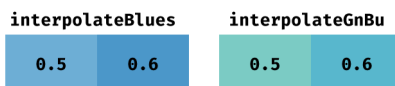
These single-hue scales are great for basic charts and charts with multiple color scales. However, sometimes the steps in between color values are too small and it becomes hard to distinguish between values. In this case, **d3-scale-chromatic** has many **sequential** color scales that cycle through another hue, increasing the difference between values.

⁵⁸<https://github.com/d3/d3-scale-chromatic>



color scales - sequential

We can see how much easier it is to tell the difference between colors 50% and 60% of the way through a single-hue scale (left) versus a multi-hue scale (right).



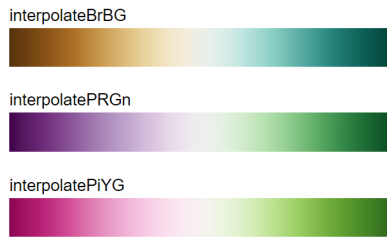
color scales - sequential - diff

3. Representing a diverging metric

Our **sequential** color scales get more visually clear as they approach the end of the scale. This is great to highlight values on the high end (for example, the highest **humidity** values will be the most visible).

Sometimes, however, we want to highlight both the lowest *and* highest metric values. For example, if we're looking at **temperature**, we might want to highlight the coldest days as a bright blue and the hottest days as a bright red.

Diverging scales start and end with very saturated/dark color and run through a less intense middle range.



color scales - diverging

We can see that we have both single-hue (per side) and multi-hue diverging scales. Again, it's helpful to cycle through more hues so users can pick up on smaller differences between metrics, but it can get overwhelming when we are already cycling through two hues. When getting a feel for color scales, try a few different options to get an idea for what will work in your specific scenario. This is not a one-size-fits-all decision.

Custom color scales

Sometimes the built-in color scales don't cut it. Maybe you have specific colors that you want to work in, or you are going for a particular tone with your chart. No worries! We can make our own scales easily using [d3-interpolate](https://github.com/d3/d3-interpolate)⁵⁹.

Let's say that we wanted to represent temperature values with a color, ranging from "cyan" to "red". There are several methods that we could use to interpolate between those two colors, including `d3.interpolateRgb()` and `d3.interpolateHsl()`.

Let's make our own scale, running from "cyan" to "tomato" — all we need to do is pass our starting and ending color as separate parameters to `d3.interpolateHcl()`.

We'll talk about the difference between the different color spaces (**rgb**, **hsl**, **hcl**) in the next section.

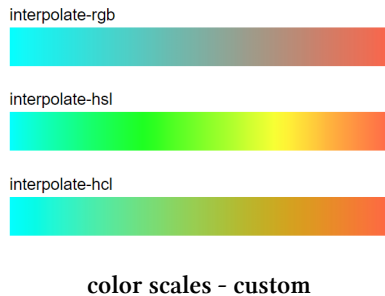
⁵⁹<https://github.com/d3/d3-interpolate>

```
d3.interpolateRgb("cyan", "tomato")
```

That will give us this scale:



Our code has been started off with a few different custom scales.



If we open up the `07-data-visualization-basics/scales/draw-scales.js` file in our text editor, we can test our own custom scales at the bottom. In the `index.html` file, we are importing other JavaScript files that list the built-in scale types and define a `drawColorRange()` function that creates an `<svg>` element with a `linear-gradient`.

We also have a utility function, `addCustomScale()`, that accepts two parameters:

1. the scale's name (which needs to be a string without spaces, since we're using it to create and reference an `id`), and
2. a color scale (a function that returns a color when passed a value between 0 and 1)

We've already created a few custom scales — check out the first three which have the same range but use different color spaces for interpolation.

We can also create new discrete color schemes. The last two custom scales are using a function called `interpolateWithSteps()` defined on line 33. `interpolateWithSteps(n)` returns a new array of `n` elements interpolating between 0 and 1. For example, `interpolateWithSteps(3)` returns `[0, 0.5, 1]`.

We can use this function to make a new color scheme by stepping through a color scale and returning equally spaced through the range. For example:

```
1 interpolateWithSteps(6).map(  
2   d3.interpolateHcl("cyan", "tomato")  
3 )
```

will create the following color scheme.



color scales - custom with steps

We can look at the color scheme by passing a unique id and our scale to `addCustomScale()`.

```
addCustomScale(  
  "interpolate-hcl-steps",  
  interpolateWithSteps(6).map(  
    d3.interpolateHcl("cyan", "tomato")  
  )  
)
```



color scales - custom - discrete

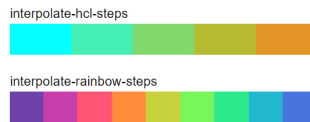
If we wanted to create discrete color schemes with many colors, we could use one of **d3-scale-chromatic**'s cyclical color scales.



color scales - cyclical

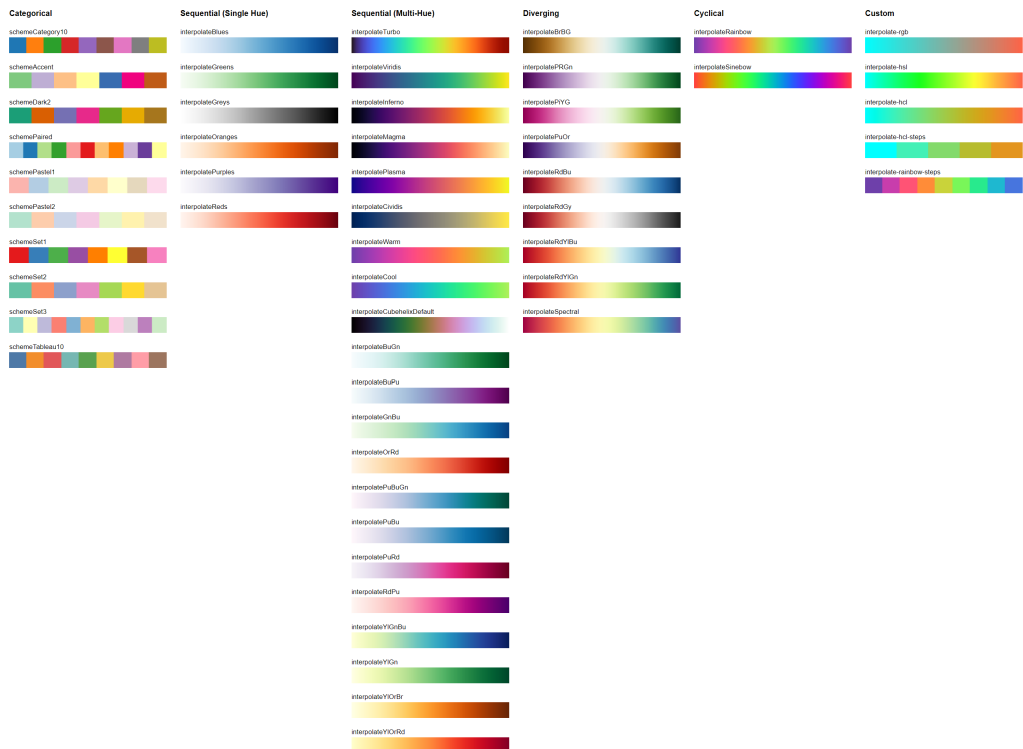
We can use our `interpolateWithSteps()` function to split this continuous color scale into a color scheme.

```
interpolateWithSteps(10).map(
  d3.interpolateRainbow
)
```



color scales - custom - discrete

Play around and make a few color scales or color schemes to get the hang of it.



color scales

Creating our own colors

Colors can be represented in various formats, each with different strengths. But creating our own colors requires care and the *color space* we use affects how well our chart can be interpreted.

Below, we're going to look at three color spaces:

- `rgb`
- `hsl` and
- `hcl`

While `rgb` is generally the most familiar, there are good reasons to use `hcl` when we're programmatically creating color scales. Read on to find out why.

keywords

In our chart code, we've used a lot of **keywords** which map to specific, solid colors. For example, we made our histogram bars **cornflowerblue**. Color keywords are great because they are easy to remember and mirror the way we refer to colors in natural language.

currentColor

Most color keywords are ways we would describe a color in English, but there is one handy special case: `currentColor`. `currentColor` will set the CSS property to the current color property. This is great for controlling one-color charts from outside a charting component. For example, we could make a line chart component whose `stroke` is `currentColor`. This way, we could easily create multiple line charts with different line colors, just by setting the `color` higher up in the DOM.

transparent

Another useful color keyword is `transparent`. This is great when creating invisible SVG elements that still capture mouse events, such as the listening rect we used to capture mouse movement in **Chapter 5**.

When creating data visualizations, we'll often need to manipulate colors. In this case, we can't use the color keywords because there is no way to make **cornflowerblue** 10% darker. In order to manipulate our colors, we'll need to learn about color spaces.

rgb

The color space you're most likely to come across when developing for the web is **rgb**. **rgb** colors are composed of three values:

r: red

g: green

b: blue

For each of these values, a higher number uses more of the specified color. Then these values are combined to create one color.

This is essentially how LCD screens work: each pixel is made up of a red, green, and blue light. The values are combined in an additive fashion, starting at black and moving towards white with higher values. This may be counter-intuitive to anyone who's used paint on paper, where more paint = a darker color. For example, `rgb(0,0,0)` represents **black** and `rgb(255,255,255)` represents **white**.

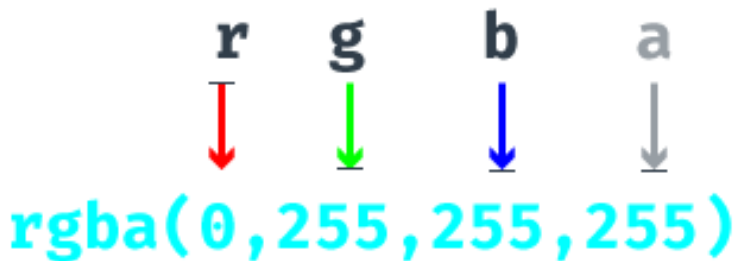
There is another optional value for all of the color spaces we'll talk about:

a: alpha

The alpha value sets the opacity of the color — a color with an alpha value of 0 will be transparent. If the alpha value isn't specified, the color will be fully opaque.

rgb can be expressed in two formats. The first, **functional notation**, starts with **rgb** and lists each color value, in order, within parentheses. Each value will be within a range from 0 to 255. If we wanted to specify an alpha value, we would switch the prefix to **rgba**.

For example, we would represent a **cyan** color as a combination of **green** and **blue** colors.



r **g** **b** **a**

↓ ↓ ↓ ↓

rgba(0, 255, 255, 255)

color space - rgb

rgb colors can also be expressed with **hexidecimal notation**, which begins with # and lists each value with two characters in a range from 00 to FF.



r **g** **b** **a**

↓ ↓ ↓ ↓

#00FFFFFF

color space - rgb as hex

rgb can be an unintuitive color space to work in — if I have a blue color and want an orange of the same brightness, all of the values have to change.

rgb(39, 204, 199)

rgb(204, 118, 41)

color space - rgb shift

Next, let's look at a color space that is closer to our mental model of color.

hsl

In the **hsl** color space, the values don't refer to specific colors — instead, they refer to color properties:

h: hue. The hue represents the angle around a circular color wheel that starts at red (0 degrees) and cycles through orange, yellow,, back around to red (360 degrees).

s: saturation. The saturation value starts at gray (0%) and ramps up to a completely saturated color (100%).

l: lightness. The lightness value starts at white (0%) and ramps up to black (100%).

(a: alpha. Again, the alpha channel is optional and defaults to full opacity (100%))

In **hsl**, our cyan color would be partially around the color wheel, fully saturated, and of medium lightness.

h s l a
↓ ↓ ↓ ↓
`hsla(180,100%,50%,100%)`

color space - hsl

hsl more closely matches our mental model of the relationship between colors — to switch from a blue to a similarly dark & saturated orange, we would only have to update the **hue** value.

`hsl(177,80,80)`

`hsl(28,80,80)`

color space - hsl shift

hcl

The **hcl** color space is similar to **hsl**, with the same values (**c** is for **chroma**, which is an alternative measure of colorfulness).

h c l a
↓ ↓ ↓ ↓
`hcl(180,100,150,100)`

color space - hcl

Instead of being spacially uniform, the values are **perceptually uniform**, so that a red and a blue with a lightness of 50% will *look* the same amount of light. Let's look at the hue spectrum at 100% saturation and 50% lightness for both **hsl** (top) and **hcl** (bottom).



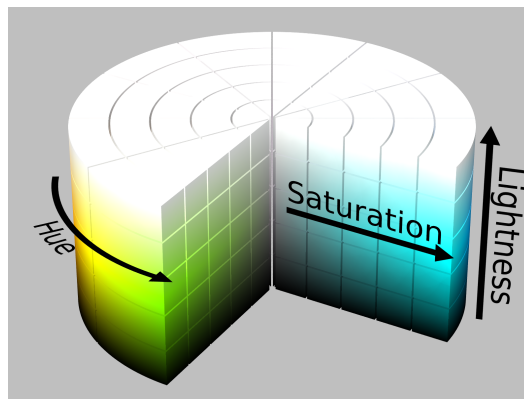
color space - hsl hues



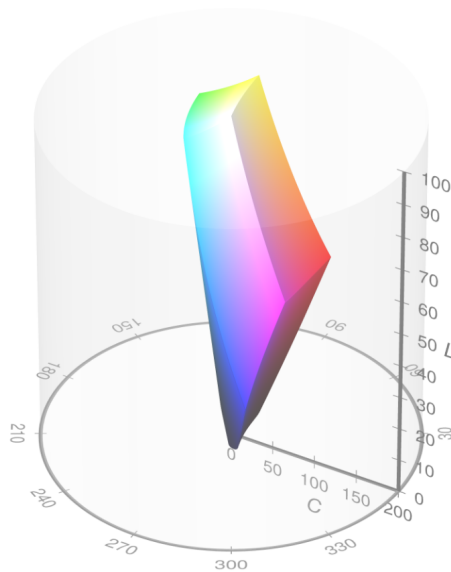
color space - hcl hues

We can see bands of color in the **hsl** hues that look lighter than the other hues (even though they mathematically aren't). Those bands are not visible in the **hcl** spectrum.

If we create a 3d cylinder of the **hsl** color space, we would have a perfect cylinder. But when we visualize the **hcl** color space this way, we can see that it's not a perfect cylinder. This is because humans can't detect changes in saturation for all colors equally.



color space - hsl 3d. From https://en.wikipedia.org/wiki/HSL_and_HSV



color space - hcl 3d. From https://en.wikipedia.org/wiki/HCL_color_space

This makes **hcl** to ideal for creating color scales — if we're using color to represent different categories of precipitation, those colors should have the same amount of visibility. This will help prevent one color from dominating and skewing our read of the data.

d3-color

While the browser will recognize colors in **rgb** and **hsl** formats, there isn't much native functionality for manipulating colors. The **d3-color**⁶⁰ module has methods for recognizing colors in the **hcl** format, for converting between formats, and for manipulating colors along the color space dimensions.

We won't go into the nitty gritty of **d3-color**, but it's good to be aware of the library. If you find yourself needing to manipulate colors manually, check out the docs at github.com/d3/d3-color⁶¹.

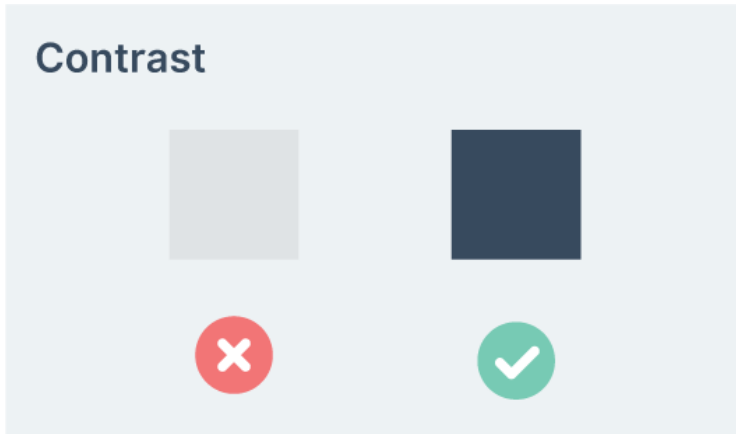
⁶⁰<https://github.com/d3/d3-color>

⁶¹<https://github.com/d3/d3-color>

Color tips

Here are a few tips to keep in mind when choosing a color scale.

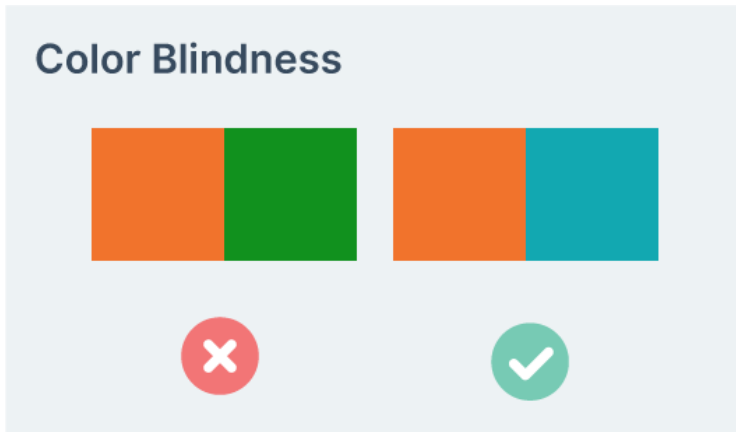
Contrast



contrast example

Make sure that your colors have enough contrast with their surroundings in all cases. If you're using Chrome, there is a great tool to check that your colors have enough contrast right in your dev tools. Learn how to use it in the Appendix.

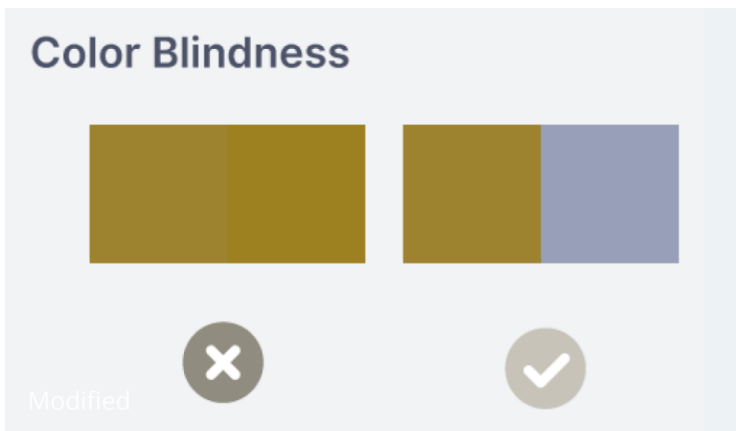
Color blindness



color blindness example

Don't assume that your users can see your charts as clearly as you can. Almost 8 percent of males of North American descent have red-green color blindness. To make sure most people can encode the information from your color scale, stay away from scales where users have to distinguish between red and green.

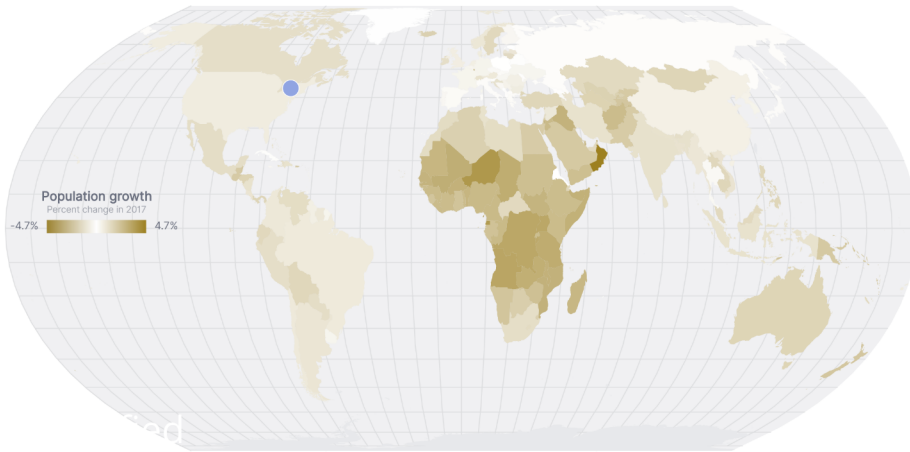
Here's a simulation of what the above picture looks for people with red-green color blindness.



color blindness simulation example

It's impossible to see which countries have negative growth and which have positive

growth if we use the red/green colors on the left for our color scale on the map we created in **Chapter 6**.



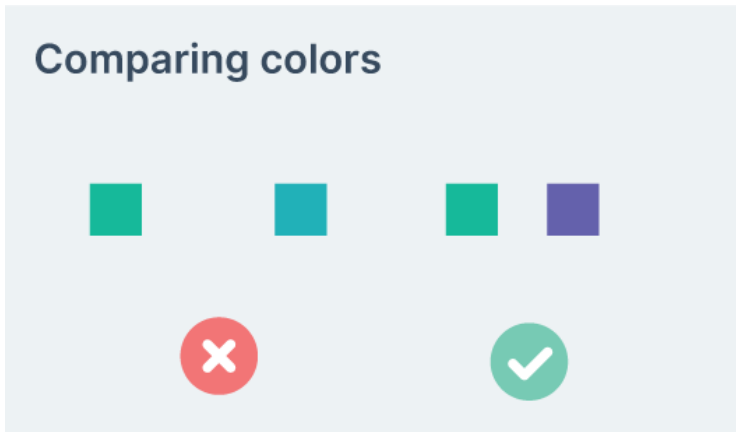
color blindness simulation map

There are other forms of color blindness (yellow-blue and total), but they are less common (under 1% of the population). When creating a chart with a color scale, it helps to have a second way to extract the information, such as adding tooltips or in-place labels.

There are sites where you can simulate different types of color blindness to test your data visualizations, such as [Color Flip](https://www.canvasflip.com/color-blind.php)⁶².

⁶²<https://www.canvasflip.com/color-blind.php>

Comparing colors

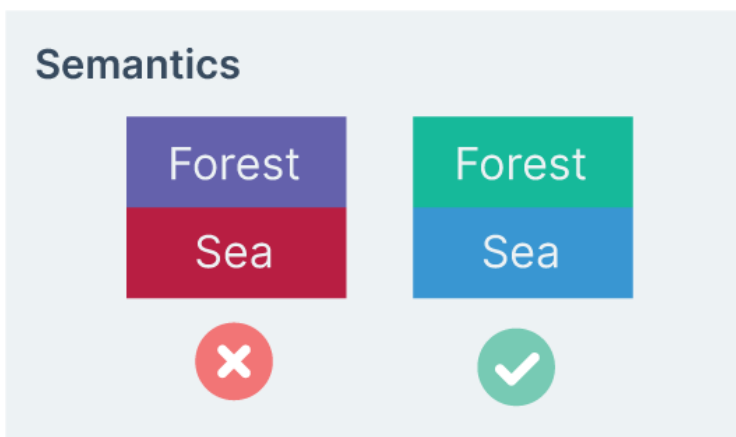


comparing colors example

Smaller areas of color need are harder to compare. When using colors as an indicator, make sure the colored elements are large enough.

Additionally, far-apart areas of color are harder to compare. Make sure the colors you use have enough contrast that users can easily tell the difference.

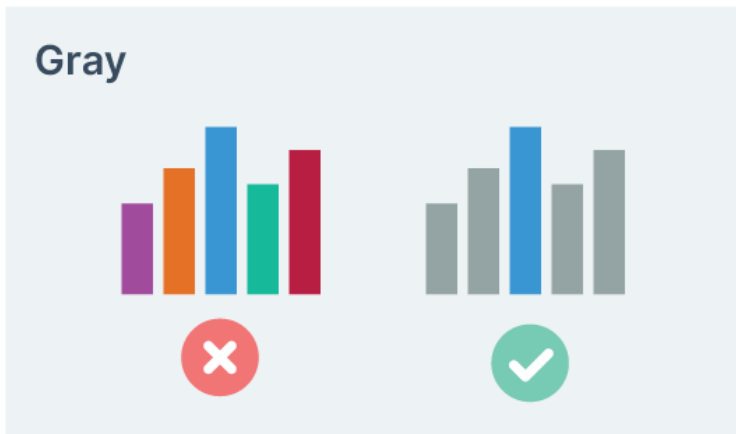
Semantics



semantics example

Choose semantically meaningful colors when possible. For example, blue for lower and red for higher temperatures.

Gray



gray example

Gray can be the most important color. When you want to highlight part of a chart, keeping other elements gray helps viewers focus on that element.

Wrapping up

Great work making it through this chapter! Fundamentals can be tedious to learn, but understanding the bare bones will give you a good foundation to making your own data visualizations.

If you want a handy way to remember these basics, we included a cheat sheet PDF with the advanced package – feel free to print it out or store it somewhere easy-to-find!

Data visualization basics cheat sheet

Types of data

Qualitative



Usually a string

Qualitative Binary



EXAMPLE
raining, not raining

Qualitative Nominal



EXAMPLE
sunny, cloudy, windy

Qualitative Ordinal



EXAMPLE
not windy, very windy

Quantitative



Usually a number

Quantitative Discrete



EXAMPLE
2 tornados, 3 tornados

Quantitative Continuous



EXAMPLE
70 degrees, 71.4 degrees

Data dimensions

Dimension Size



EXAMPLE
small, medium, large

Dimension Position



EXAMPLE
left, center, right

Dimension Color



EXAMPLE
red, green, blue

Data visualization tips

Contrast



EXAMPLE
light gray, dark gray

Color Blindness



EXAMPLE
red, green, blue, yellow

Comparing colors



EXAMPLE
green, teal, blue, purple

Semantics



EXAMPLE
Forest, Sea

Gray



EXAMPLE
red, green, blue

FULLSTACK D3
and DATA VISUALIZATION
AMELIA WATTENBERGER



Cheat sheet

Common charts

Let's talk about common chart types. This chapter can help you in a few ways:

Communication

When discussing part of your interface, it's important to have commonly understood names for things. Imaging trying to talk about a button without knowing the word "button".

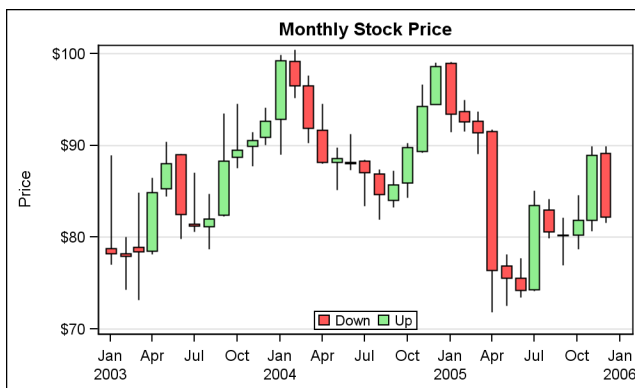
Having a common language for charts makes it easier to talk about existing and potential charts.

Quick readability

Every time a user sees a chart, they need to orient themselves to figure out how to read it. There are ways to help speed up that process, such as adding labels and legends. But the biggest help is using a familiar chart type.

Since most people have encountered a basic timeline before, showing data in a timeline will help users who don't want to spend much time learning a new visualization.

Common charts also have implicit connotations. For example, candlestick charts are often associated with stock data — an association that can speed up (or hinder) your users, depending on the usage.



Candlestick chart from blogs.sas.com/content/graphicallyspeaking/2014/09/27/candlestick-chart/

Inspiration

Common charts are a great jumping-off point for a more complex or unique chart. Only making the basic chart types can seem boring, but thinking about a dataset in the frame of one of these charts helps with brainstorming. What insights might your readers glean if you plotted your data as a histogram? What about a radar chart?

Additionally, sometimes the most basic chart is the best suited for a new feature.

Chart types

Let's go through some of the common types and discuss examples of each.

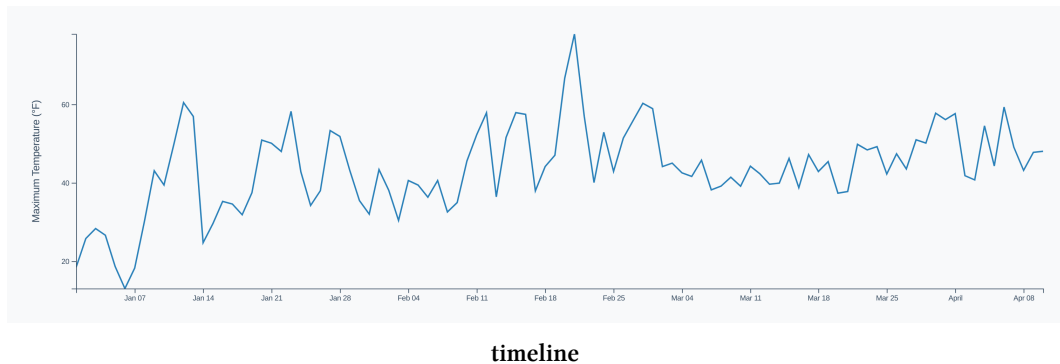


Each of these chart types has corresponding code in the `/code/08-common-charts/` folder. Be sure to check them out live in the browser and dig through the code. I guarantee you'll learn a lot from going through each example.

Please use the code as a resource — next time you want to implement a similar chart, the code will be here to guide you, using the same framework we've used throughout this book.

Timeline

Most dashboards have at least one timeline.



A timeline shows a **continuous** metric (y axis) across time (x axis).

For each of these charts, we'll practice breaking it down into its component parts — what kinds of metrics are we visualizing and in what dimensions are they visualized?

Dimensions: - **vertical position** of a point on the line represents a **continuous** metric value - **horizontal position** of a point on the line represents a **continuous** metric value (**time**)

They are very useful for showing trends over time. For example: has this stock value increased over the past year? Are there weekly patterns in this stock value?

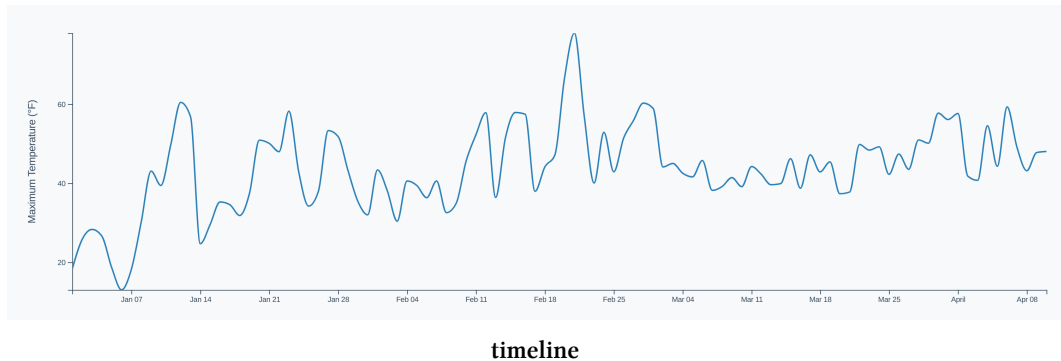
An important question to answer when creating a timeline is: what granularity do I want to see? A timeline that goes through one point per day can be too noisy when looking at yearly data. It's hard to pick up an overall trend when looking at a spikey chart.

Alternatively, a large granularity can mask small outliers — showing monthly points can hide a day with a large value.

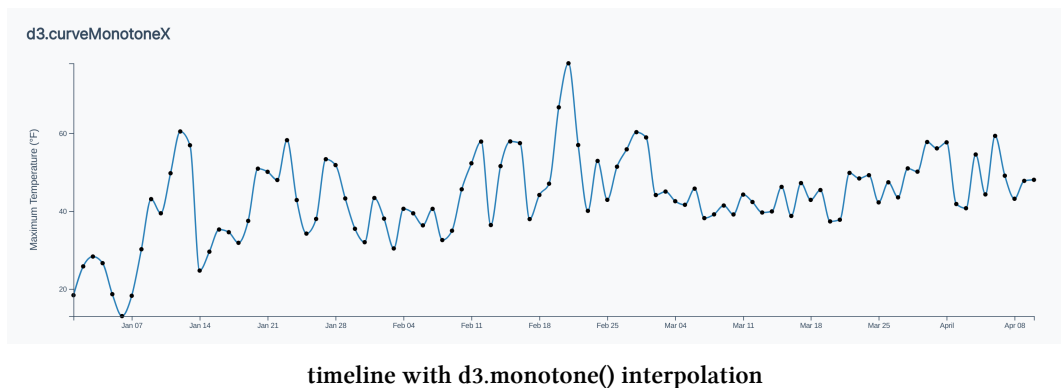
Another question to ask when creating a timeline is: how smooth do I want my line to be? If your audience wants to know the exact points of a timeline, stick with a

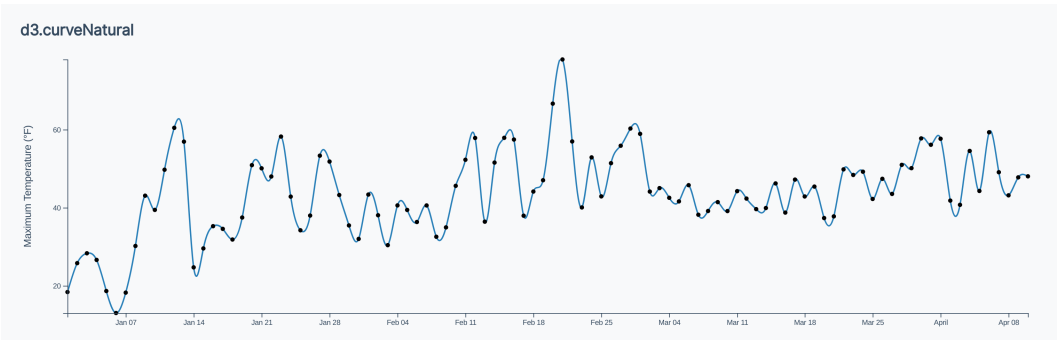
linear interpolation (the default for `d3.line()`).

A smoother line can help with showing the bigger picture — in a sense, we're guessing what the missing values are between points.

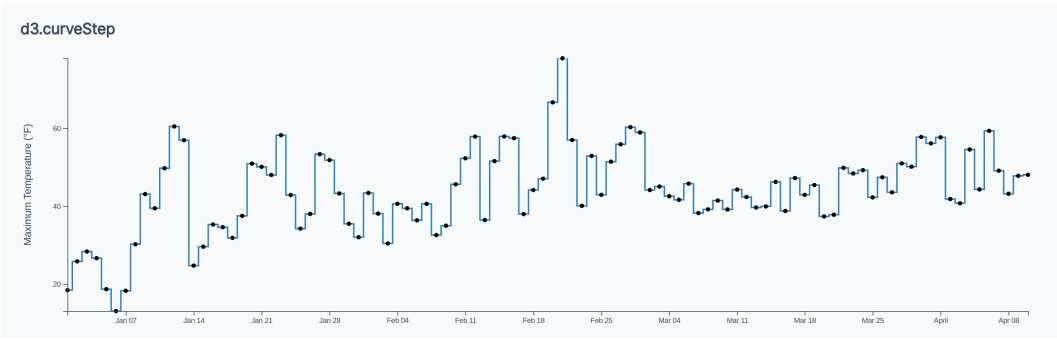


We can change the smoothness by using different interpolations and passing them to `d3.line().curve()`. Here is our line with different interpolations (the original points are visualized as dots):

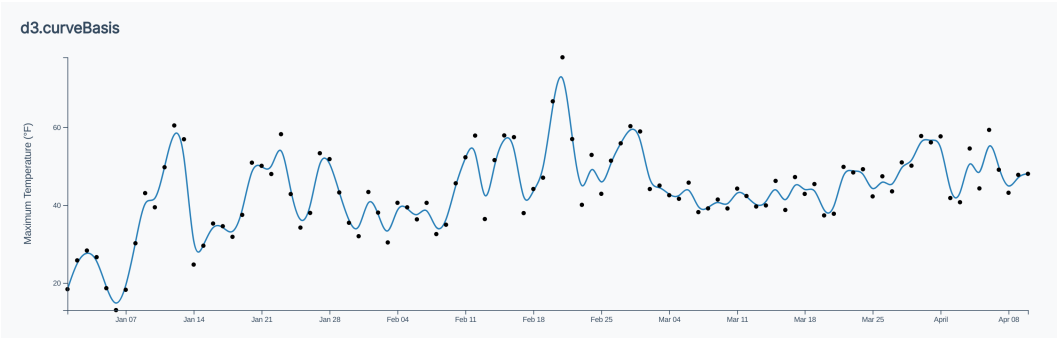




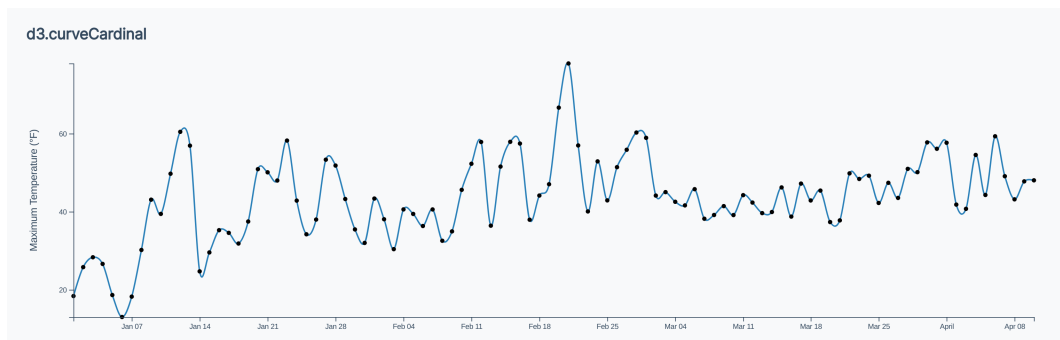
timeline with d3.curveNatural() interpolation



timeline with d3.curveStep() interpolation



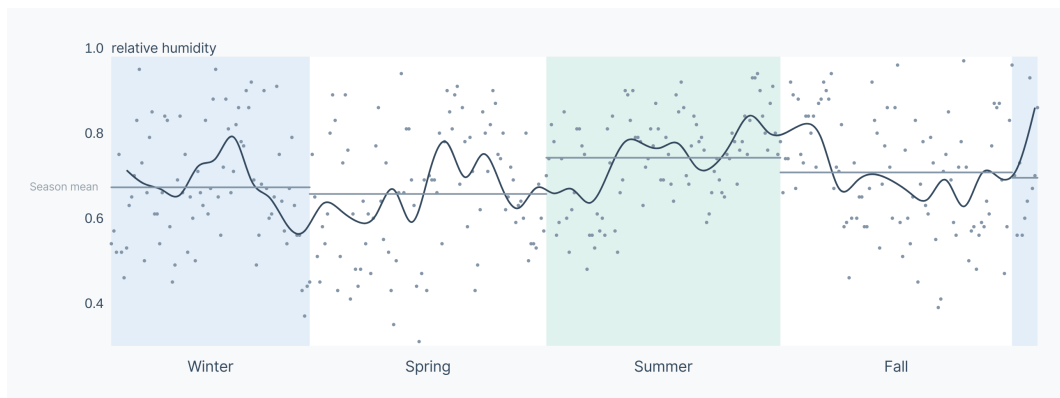
timeline with d3.curveBasis() interpolation



timeline with `d3.curveCardinal()` interpolation

In general, `d3.curveMonotoneX()` is a good bet since it crosses through each point. With other interpolations, such as `d3.curveBasis()`, the line will “skip” points to create a smoother line, which can make adding tooltips tricky, and also won’t ensure that your line will stay within your bounds.

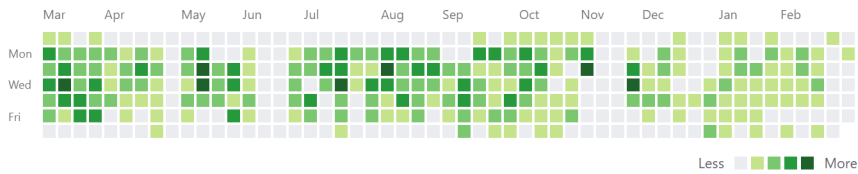
One thing to be cautious about is lying with data — make sure you aren’t misleading the reader into thinking that you have more data points than you do. One way of denoting a *trend* is by showing the original points as well as a trend line, like we did in the last chapter.



Humidity timeline, finish

Heatmap

You might be most familiar with heatmaps from the Github contribution graph.

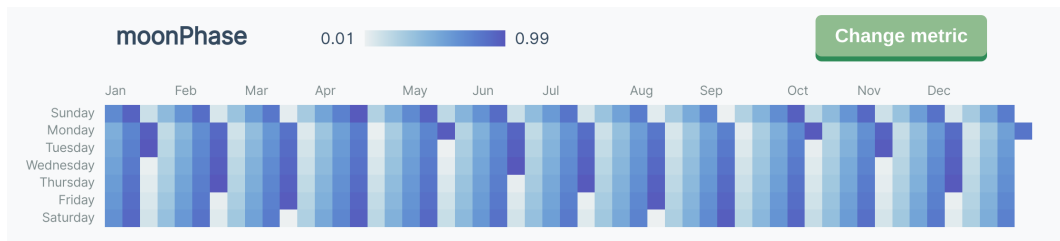


github heatmap

A heatmap has three dimensions: x, y, and a color scale. In this case, the x axis corresponds to week, the y axis corresponds to day of the week, and the color scale domain is the number of contributions per day.

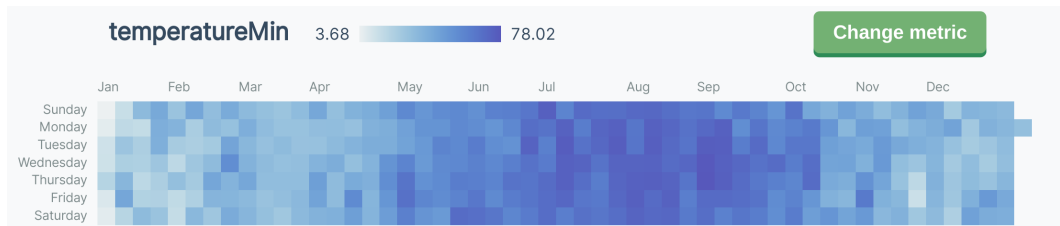
Dimensions: - **vertical position** of a square represents the day of the week (**discrete**) - **horizontal position** of a square represents the week of the year (**discrete**) - **color** of a square represents a **continuous** or **discrete** metric value (often using a **sequential color scale**)

In our code library's heatmap, we can cycle through different metrics to represent in our color scale. For example, the moon phase heatmap shows a very clear 27-day cycle.



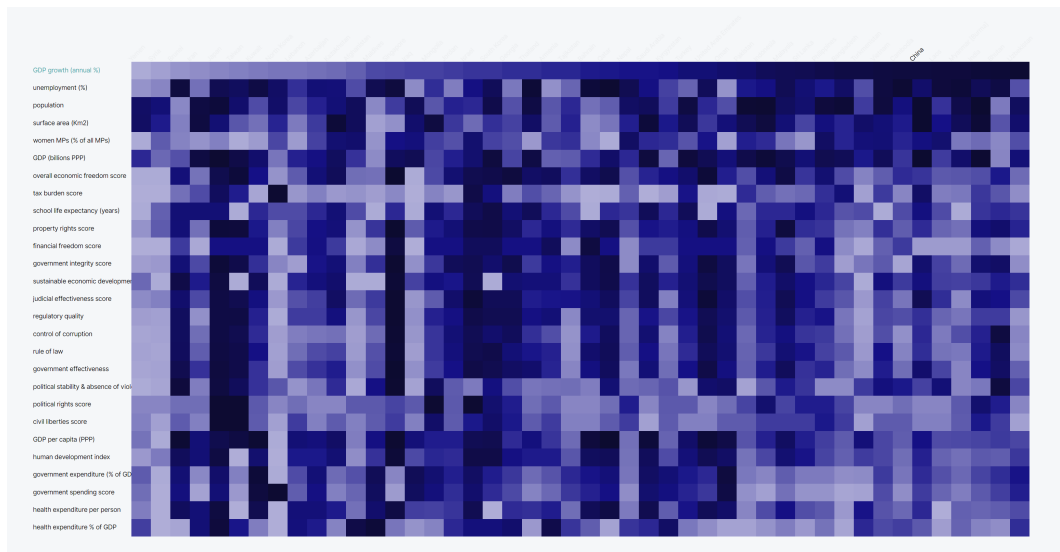
heatmap of moon phase

Alternatively, if we look at the minimum temperature heatmap, we can see the much more gradual change of temperature between the four seasons.



heatmap of min temperature

Heatmaps don't necessarily need to show one metric over time. For example, the following heatmap shows percentile for Asian countries (x axis) for different metrics (y axis). Note that heatmap cells don't necessarily need to be square.

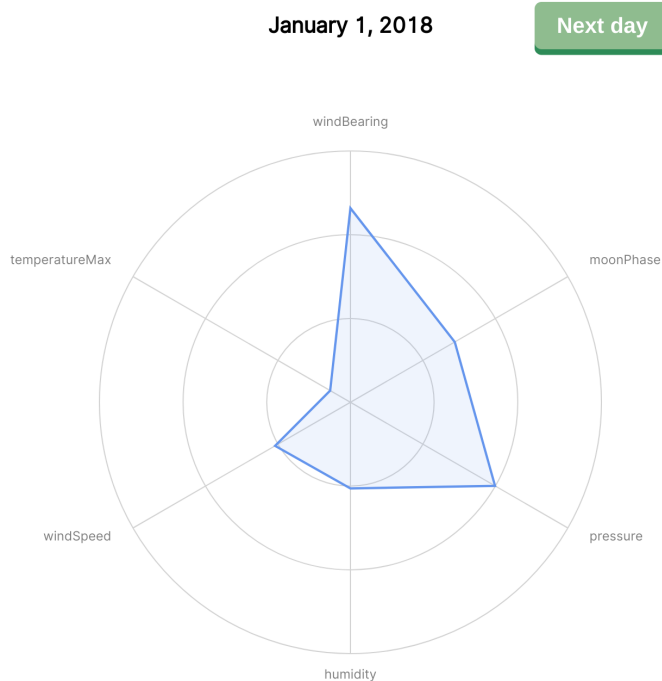


country metrics heatmap

Heatmaps are great at showing trends over time and correlations between metrics. It's harder to tell small differences in color scales than, say, bar sizes, so stay away from them or add informative tooltips if you need to show exact numbers.

Radar

Radar charts are not the most common chart type, but they can be very useful when looking at multiple metrics.



radar chart

They consist of at least three axes, where each axis represents one metric.

Dimensions: - **position** of a point on the line on each spoke represents a **continuous** or **discrete** metric value

While they aren't very helpful for making exact measurements (it's hard to tell how much larger variable A is on chart 1 versus chart 2), they really shine when comparing many charts with many metrics. Each chart creates a *profile* of that instance, and multiple *profiles* can be easily compared to find similarities and outliers.

There are a few ways to present multiple *profiles*:

1. **Show multiple polygons of different colors, all on the same chart.** This works well when there are fewer than five *profiles*.
2. **Show one chart with a toggle between *profiles*.** This works well when you want to focus on one *profile* at a time, potentially with more information alongside the chart.

3. Show multiple radar charts. This works well when comparing more than five *profiles*, with no extra information.

If we look at a group of radar charts for our weather data, we can pick out patterns.



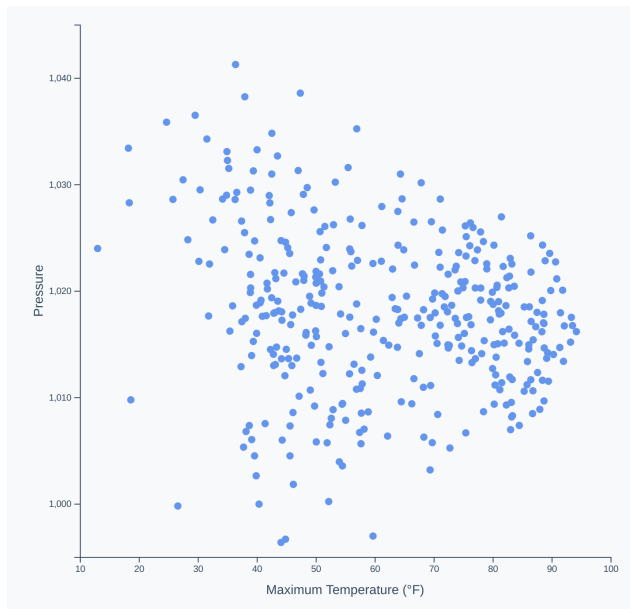
radar charts, small multiples

For example, days in June have a common pattern (up and to the right) and days in early November have a different pattern (mostly even all around). Additionally, we can see that November 4th is a bit of an outlier, with an unusually low wind speed.

These patterns seem obvious, given what we know about seasons — winter days have different weather than spring days. But we can dive in deeper to see more granular patterns — for example, the second set of days are windier and less humid. Additionally, we know a lot about how the weather changes between seasons. Imagine looking at radar charts for less familiar data — radar charts would help us quickly pick up patterns across many metrics.

Scatter

Scatter plots are great for looking at the relationship between two metrics (x and y axes).



scatter plot

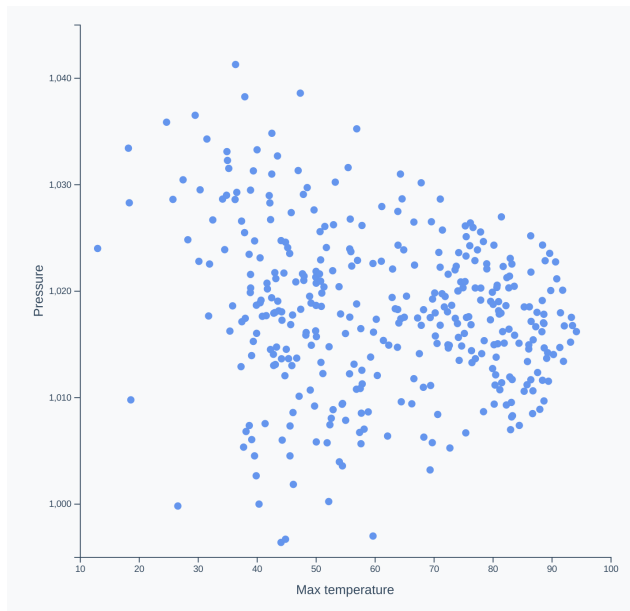
Dimensions: - **horizontal position** of a dot represents a **continuous** metric - **vertical position** of a dot represents a **continuous** metric

They work best when the chart is square, giving the same **range** to both metrics.

There are three basic patterns to look for in a scatter plot.

Positive

The two metrics are correlated and increase together in a linear fashion.

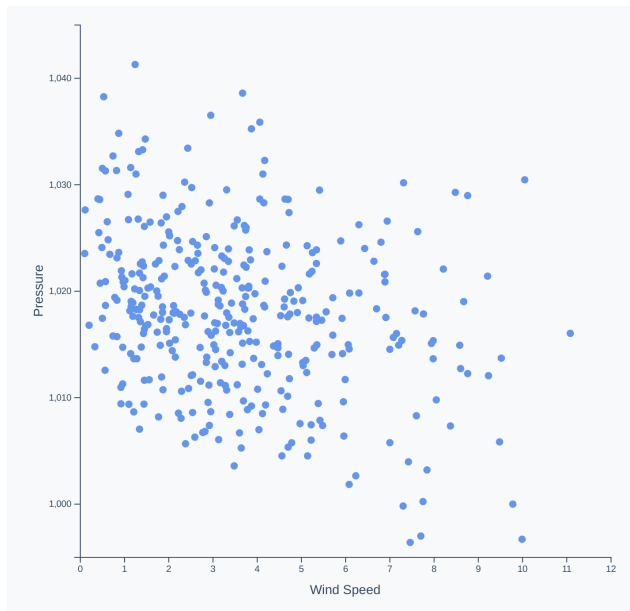


scatter plot, positive

For example, a scatter plot of minimum temperature and maximum temperature will have a positive pattern, since temperature varies much more across days than within days.

Negative

The two metrics are anti-correlated and one decreases while the other increases.

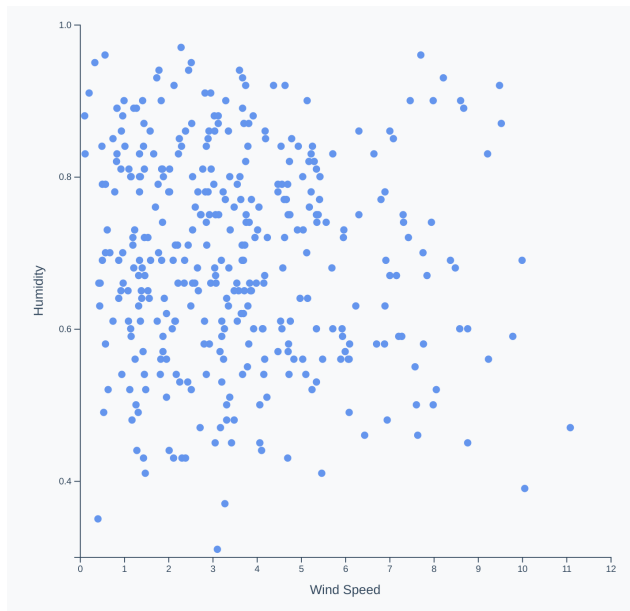


scatter plot, negative

For example, a scatter plot of wind speed and pressure will have a negative pattern, since windier days tend to have less more pressure.

Null

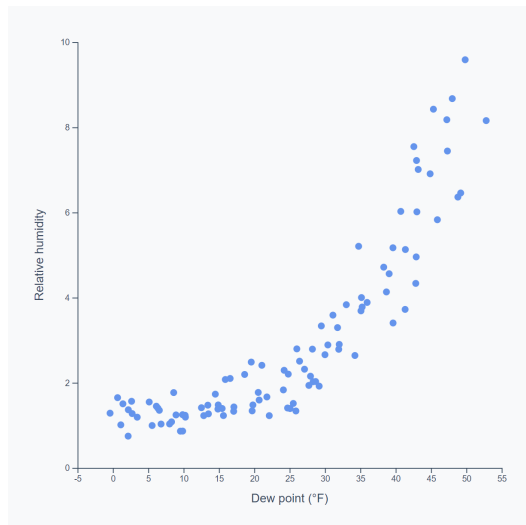
The two metrics are not related.



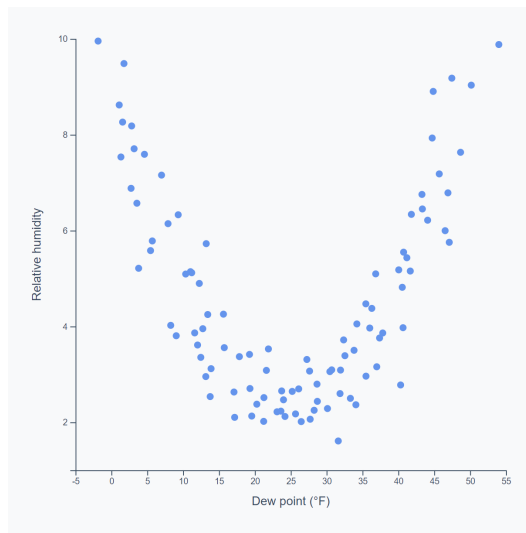
scatter plot, null

For example, a scatter plot of wind speed and humidity will have no pattern, since wind speed and humidity are not at all correlated.

There are many, more complex patterns that might be visible in a scatter plot. For example, scatter plots can be **exponential** or **u-shaped**.



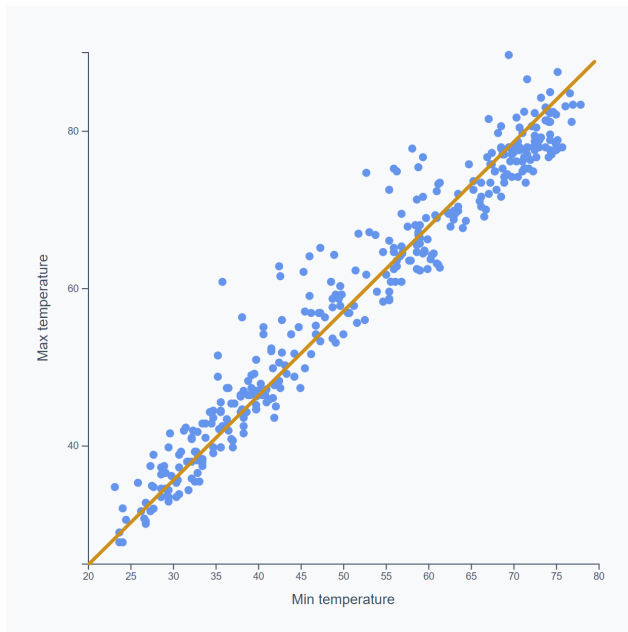
scatter plot, exponential



scatter plot, u-shaped

These kinds of patterns are less common and harder to interpret.

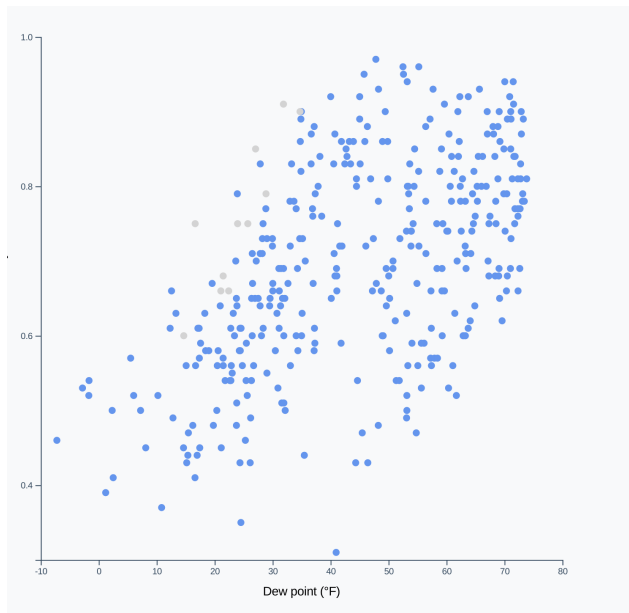
A scatter plot might also have a **line of best fit** that is the best guess of how the two metrics are correlated.



scatter plot with line of best fit

When looking at a scatter plot, keep one of the golden rules of data analysis in mind: **correlation does not imply causation**. Even if a scatter plot has a strong positive pattern, you cannot conclude that metric A causes metric B, or vice versa.

Sometimes an extra dimension is added to a scatter plot — a color scale or different shapes per category. For example, we gave different kinds of precipitation a different color in **Chapter 2**.

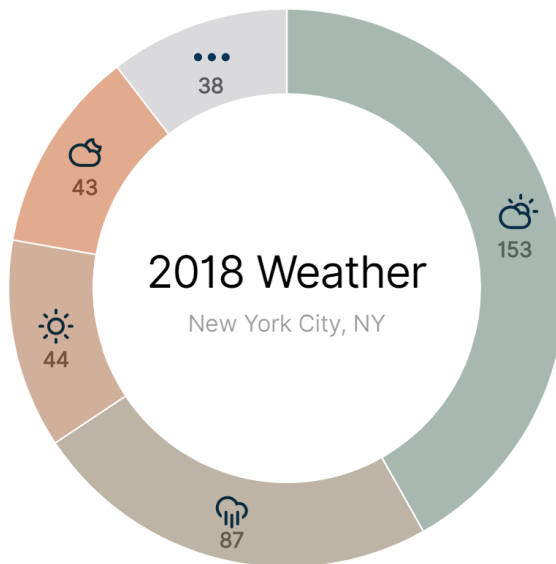


scatter plot with precipitation

This can help identify **groups** within your dataset.

Pie charts & Donut charts

Here is one of the most controversial types of charts — the pie chart! A pie chart and a donut chart are very similar — a donut chart is just a pie chart with the middle cut out.



Icons from [Climacons](#) by Adam Whitcroft

donut chart

Dimensions: - angle of a slice represents a **continuous** or **discrete** metric

I bet you've heard lots of opinions about pie charts. Part of the controversial stigma comes from their overuse in inappropriate situations, or the popularity of showing them in 3d. I have yet to find an appropriate use for a 3d pie chart.

That being said, pie charts can be useful — mainly because they are intrinsically tied to the idea of “parts of a whole”. We've all cut a pie or a pizza into multiple parts before, while focusing on creating equally-sized slices. When a user sees a pie chart, it's immediately obvious that they are looking at the composition of a whole.

However, because of human perception, it's way easier to compare bar lengths than slice sizes. In most cases a pie chart is used, another chart type would have been more effective. Here are a few tips for making effective pie charts:

- Restrict the number of slices to five or fewer. Note that we combined the last

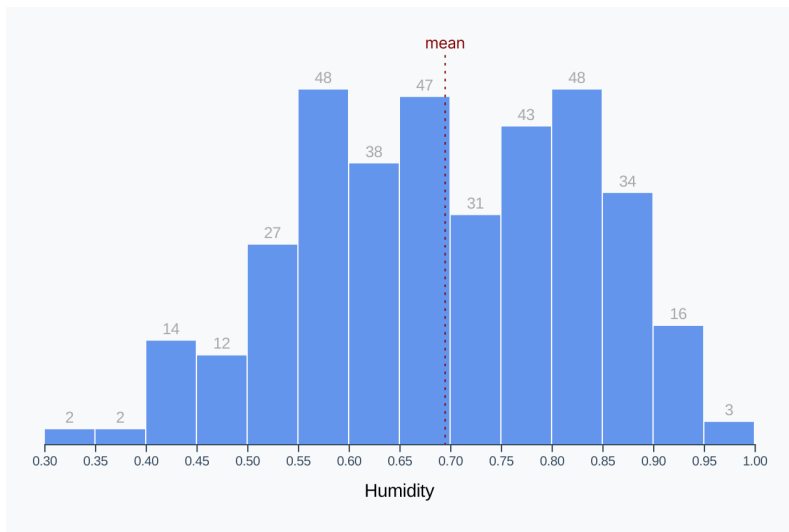
few slices into an “other” category to keep the focus on the larger slices.

- Order the slices by size
- Label each slice directly

Make sure to check out the code example in `/code/08-common-charts/pie/` folder to see d3’s arc-drawing functionality in action!

Histogram

We’re already familiar with this next chart type — we made our own histograms in **Chapter 3**. Histograms are unique in that they are only concerned with one metric.



histogram

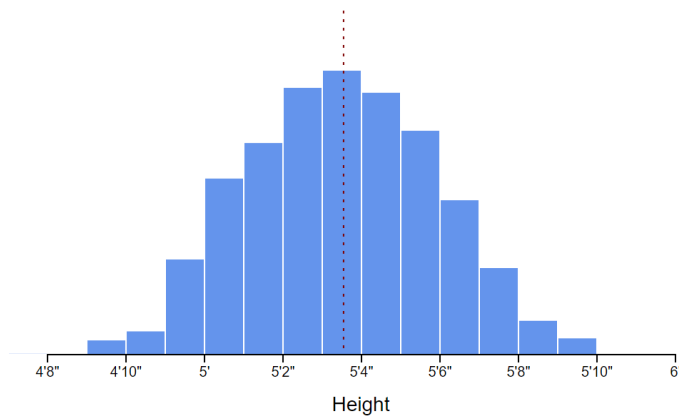
Dimensions: - **horizontal position** of each bar represents a **discrete** metric (usually a binned **continuous** metric) - **height** of the bar represents a **discrete** metric (count of the data points that fall in the bucket)

Use a histogram when you want to look at the variation of values for a single metric. Do we have mostly high or low temperatures, or does temperature vary closely around one value?

There are five basic types of distributions:

Normal

A histogram with a normal distribution is centered around a mean. Usually, this data . Most metrics that rely on chance end up with a normal distribution, such as per-sex height.

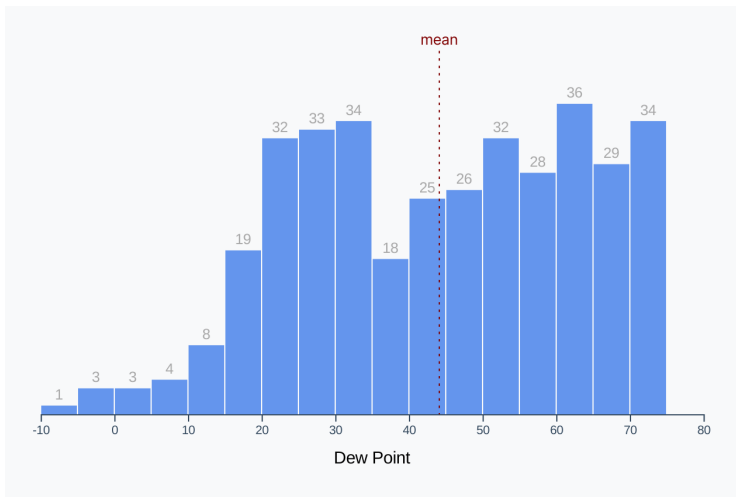


histogram

For example, the range of heights in an all-female class is likely to be a normal distribution.

Skewed right

In a histogram that is skewed to the right, a value is more likely to lie in the upper half of values.

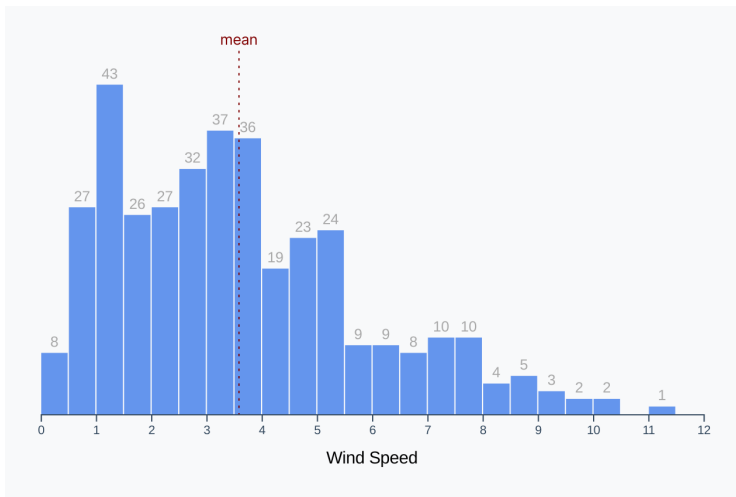


histogram - dew point

We saw an example of a skewed right histogram in **Chapter 3** when we looked at dew point. This histogram is only slightly skewed to the right, but you can see the tail off to the left.

Skewed left

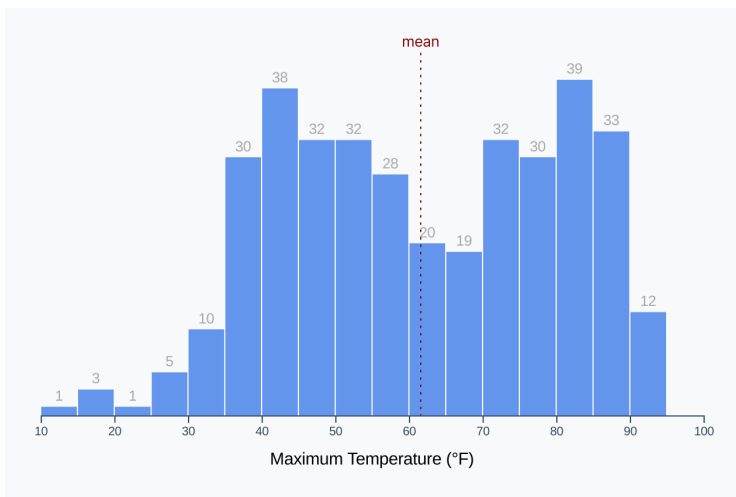
The opposite of **skewed right**, in a histogram that is skewed to the left, a value is more likely to lie in the lower half of values. This often happens when a metric has a lower bound, since values can't dip below a certain number.



histogram - wind speed

Bimodal

A bimodal histogram has two clear peaks.

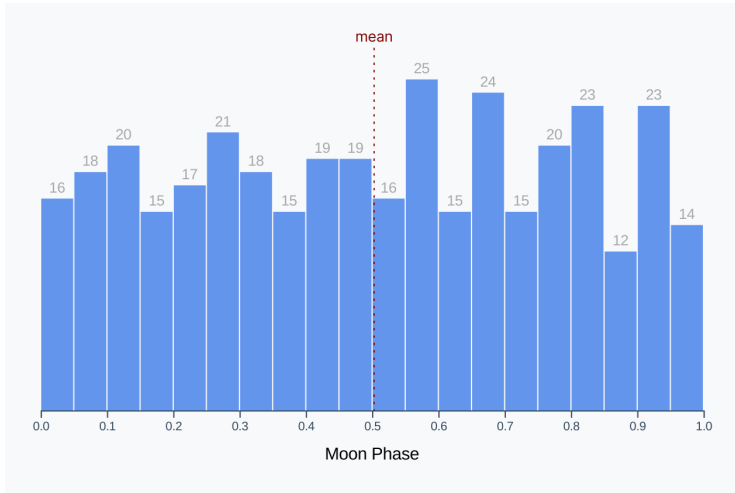


histogram - min temperature

We saw an example of a bimodal histogram in **Chapter 3** when we looked at max temperature. The daily temperature seems to either be around 45 degrees or 80 degrees - representing the extremes of winter and summer.

Symmetric

A symmetric histogram no clear peaks and looks fairly flat.



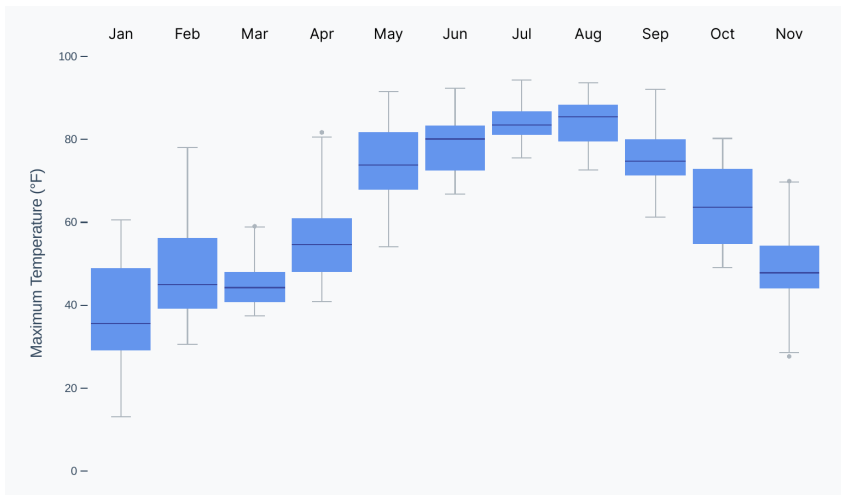
histogram - moon phases

We saw an example of a symmetric histogram in **Chapter 3** when we looked at moon phase. The moon phase is cyclic, with the value incrementing equally every day, so every value is equally as likely.

Try to revisit your histograms from **Chapter 3** and see which category each of your metrics' distributions falls under.

Box plot

Box plots are great for showing discrete or binned continuous metrics — usually over time or relative to another metric.



box plot

Each **box** shows the distribution of values for a category. For example, the first **box** in this chart shows the range of all humidity values in January.

Dimensions: - **horizontal position** of each box represents a **discrete** metric - **vertical position** of each box and whisker represents a **continuous** metric - **height** of each box and whisker represents a **continuous** metric

The middle line represents the median (the middle value) and the box covers values from the 25th percentile (25% of values are lower) to the 75th percentile (75% of values are lower). The “whiskers” extend 1.5 times the inter-quartile range (**IQR**) from the median in either direction. By **IQR**, we mean the difference between the 25th and 75th percentiles. Any outliers that lie outside of the whiskers are shown as dots.